

5.4	Test Methodologies	10
5.5	Performance Evaluation	10

LIST OF TABLES

3.1	Functional Requirements Analysis matrix	6
4.1	Database schema definitions	8
5.1	Test case suite	10

LIST OF FIGURES

4.1	High-level System Architecture	7
4.2	Entity-Relationship Diagram	8
4.3	Data Flow Diagram	8

1 INTRODUCTION

1.1 Introduction

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

1.2 Problem Statement

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

1.3 Scope of Research

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

1.4 Research Hypothesis

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

1.5 Objectives

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

1.6 Organization of the Report

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

2 LITERATURE REVIEW

2.1 Background

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

2.2 Summary of Literature Review and Research Gap

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

3 SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Requirements Gathering

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

3.2 Feasibility Study

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

3.3 Functional Requirements

Table 3.1: Functional Requirements Analysis matrix

ID	Requirement	Priority
FR1	Auth	High

3.4 Non-Functional Requirements

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

4 SYSTEM DESIGN

4.1 Architecture

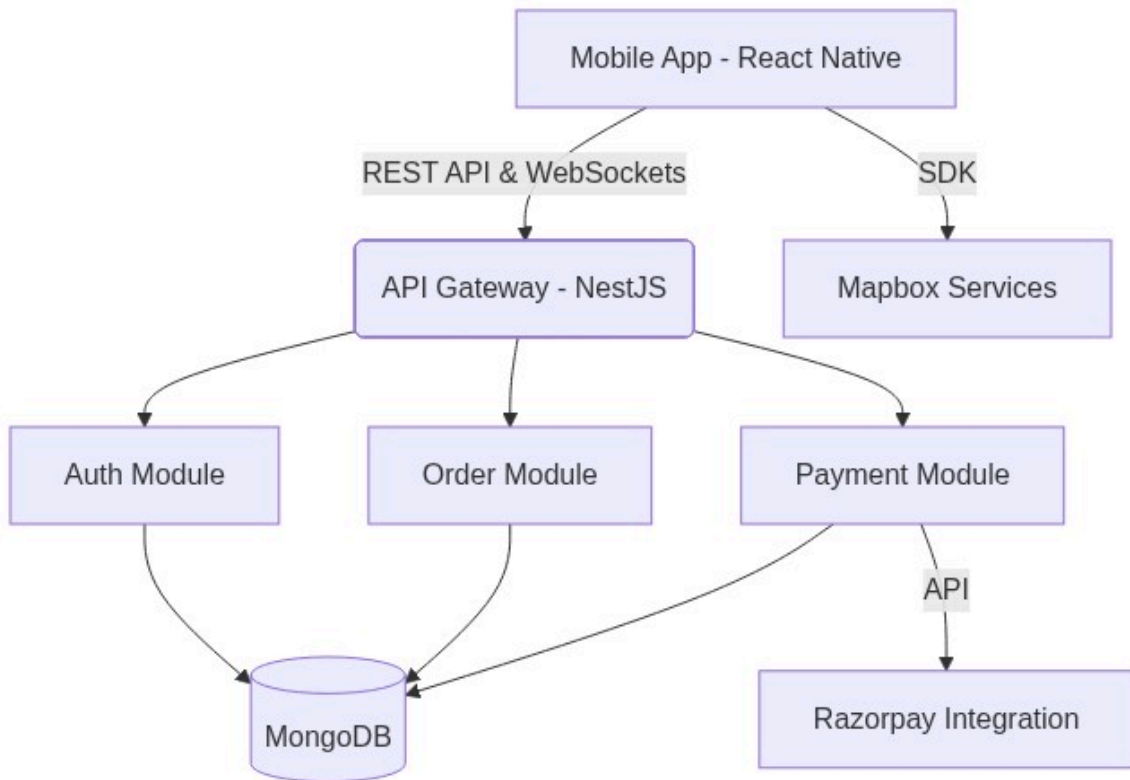


Figure 4.1: High-level System Architecture

4.2 Database Schema

Table 4.1: Database schema definitions

Collection	Description
Users	User credentials

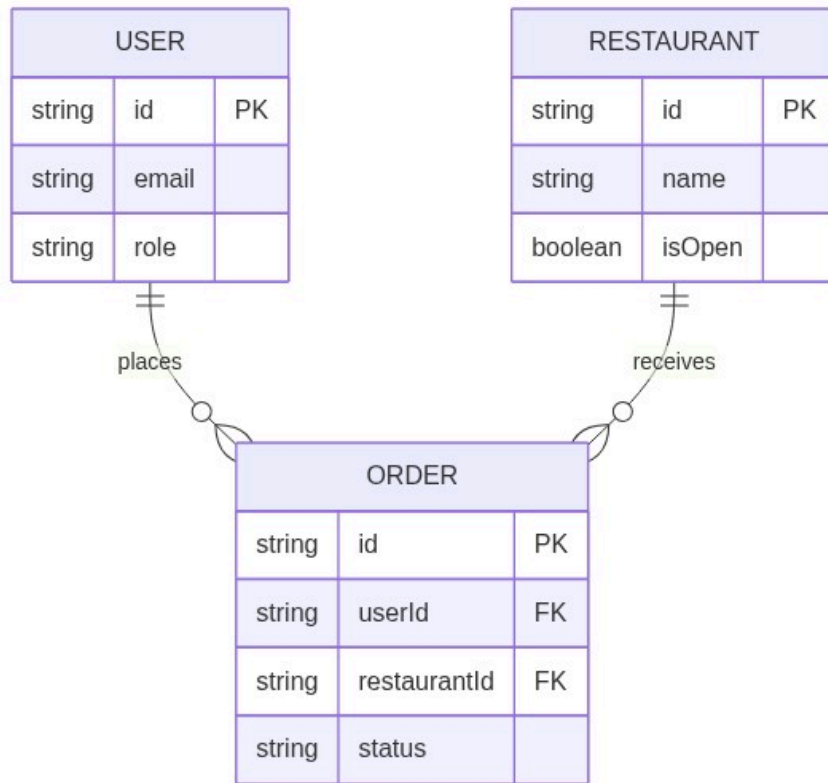


Figure 4.2: Entity-Relationship Diagram

4.3 Data Flow Diagrams

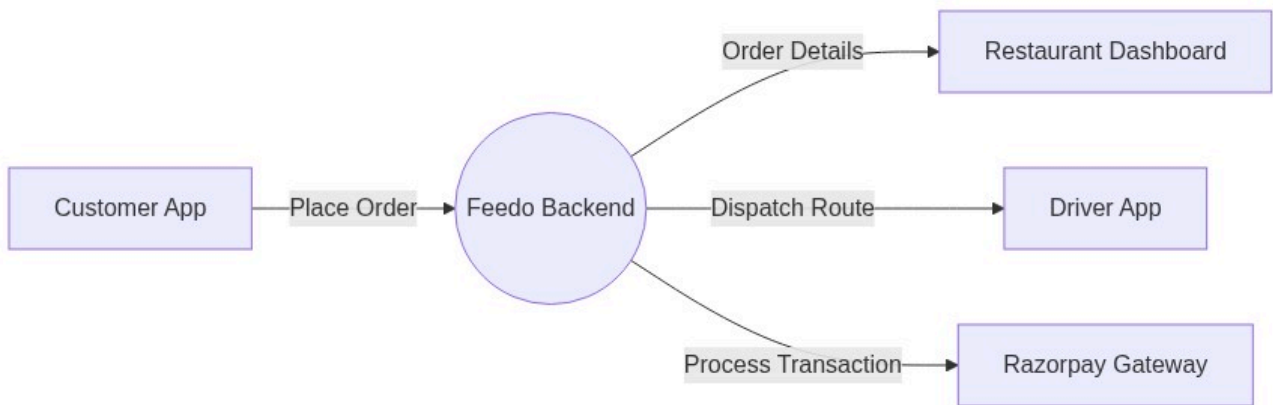


Figure 4.3: Data Flow Diagram

5 IMPLEMENTATION, TESTING AND RESULTS

5.1 Frontend Implementations



[Customer Home]

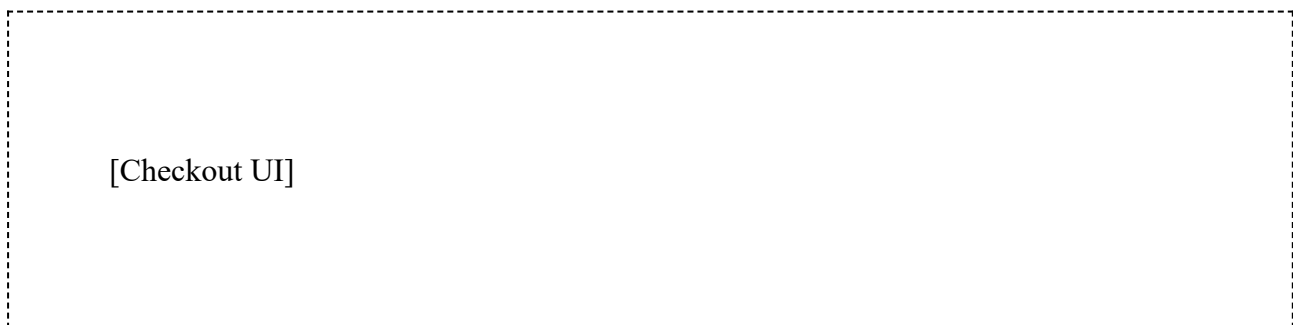
Figure 5.4: Implementation screenshot



[Cart]

Figure 5.4: Implementation screenshot

5.2 Payment Integration



[Checkout UI]

Figure 5.4: Implementation screenshot

5.3 Live Tracking Implementation



Figure 5.4: Implementation screenshot

5.4 Test Methodologies

Table 5.1: Test case suite

ID	Scenario	Result
TC1	Payment	PASS

5.5 Performance Evaluation

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location

of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

REFERENCES

[1] React Native... [2] NestJS... [3] MongoDB...